João Pedro De Souza Lopes

Projeto e Simulação de uma ULA RISC-V em VHDL

Brasília – DF
22/10/2026

João Pedro De Souza Lopes

Projeto e Simulação de uma ULA RISC-V em VHDL

Relatório apresentado à disciplina de Arquitetura e Organização de Computadores como parte dos requisitos de avaliação.

Universidade de Brasilia
Engenharia de Computação

Orientador: Professor(a): Ricardo Jacobi

Brasília – DF
22/10/2026

Sumário

1 INTRODUÇÃO 3

2 OBJETIVOS 4

3 DESCRIÇÃO DA IMPLEMENTAÇÃO 5
3.1 Operações implementadas 5
3.2 Funcionamento do sinal `cond` 5

4 COMPARAÇÕES COM E SEM SINAL 6

5 TESTBENCH E METODOLOGIA DE TESTES 7
5.1 Resultados esperados 7
5.2 Formas de onda 8
5.3 Análise dos resultados 9

6 DETECÇÃO DE OVERFLOW NAS OPERAÇÕES ADD E SUB . . 10
6.1 Overflow na soma 10
6.2 Overflow na subtração 10

7 CONCLUSÃO 11

REFERÊNCIAS 12

A CÓDIGO-FONTE DA ULA 13

B CÓDIGO-FONTE DO TESTBENCH 17

1 Introdução

Este trabalho tem como objetivo desenvolver e verificar uma Unidade Lógica e Aritmética (ULA) de 32 bits baseada nas operações especificadas no projeto RISC-V, utilizando a linguagem de descrição de hardware VHDL.

A ULA recebe dois operandos de 32 bits, denominados A e B, e um código de operação (*opcode*) que determina a operação a ser executada. O resultado é apresentado na saída Z. Adicionalmente, a saída `cond` indica se uma condição de comparação é verdadeira.

A implementação foi desenvolvida em VHDL utilizando a biblioteca `numeric_std`, permitindo realizar operações aritméticas e comparações com e sem sinal.

2 Objetivos

Os objetivos do trabalho são:

- Implementar uma ULA de 32 bits em VHDL.
- Implementar as 14 operações especificadas no enunciado.
- Utilizar o sinal `opcode` para selecionar a operação.
- Implementar o sinal `cond` para indicar o resultado das comparações.
- Desenvolver um *testbench* para verificar o funcionamento da ULA.
- Analisar as formas de onda produzidas durante a simulação.
- Explicar as diferenças entre comparações com e sem sinal.
- Apresentar métodos de detecção de *overflow* nas operações ADD e SUB.

3 Descrição da implementação

A ULA possui duas entradas de dados, A e B, uma entrada de controle, `opcode`, uma saída de dados, Z, e uma saída de condição, `cond`.

A entrada `opcode` possui quatro bits, permitindo selecionar as operações definidas na especificação.

3.1 Operações implementadas

Tabela 1 – Operações implementadas na ULA

Código	**Operação**	**Descrição**
0000	ADD	Soma dos operandos
0001	SUB	Subtração de B de A
0010	AND	Operação lógica AND bit a bit
0011	OR	Operação lógica OR bit a bit
0100	XOR	Operação lógica XOR bit a bit
0101	SLL	Deslocamento lógico à esquerda
0110	SRL	Deslocamento lógico à direita
0111	SRA	Deslocamento aritmético à direita
1000	SLT	Comparação menor que, com sinal
1001	SLTU	Comparação menor que, sem sinal
1010	SGE	Comparação maior ou igual, com sinal
1011	SGEU	Comparação maior ou igual, sem sinal
1100	SEQ	Comparação de igualdade
1101	SNE	Comparação de desigualdade

Fonte: Elaboração própria com base no enunciado da atividade.

3.2 Funcionamento do sinal `cond`

O sinal `cond` indica o resultado das operações de comparação. Quando a condição avaliada é verdadeira, `cond` recebe o valor lógico 1; quando é falsa, recebe 0. Nas operações que não são comparações, permanece em 0.

Nas operações de comparação, o bit menos significativo de Z também indica o resultado: Z recebe 1 quando a condição é verdadeira e 0 quando é falsa.

A interface especificada não possui uma saída independente chamada `zero`. Portanto, um resultado aritmético igual a zero pode ser identificado verificando se todos os bits de Z são iguais a 0.

4 Comparações com e sem sinal

Os números de 32 bits podem ser interpretados de maneiras diferentes, dependendo de serem considerados com sinal ou sem sinal.

Nas comparações com sinal, os números são interpretados como valores inteiros em complemento de dois. Dessa forma, o bit mais significativo indica o sinal do número: 0 para valores não negativos e 1 para valores negativos.

Nas comparações sem sinal, todos os 32 bits representam a magnitude de um número não negativo.

Por exemplo, o padrão hexadecimal `FFFFFFFF` representa -1 quando interpretado como inteiro com sinal de 32 bits. Quando interpretado sem sinal, representa $4\,294\,967\,295$.

Considerando $A =$ `FFFFFFFF` e $B =$ `00000001`:

- Na operação SLT, a comparação $-1 < 1$ é verdadeira.
- Na operação SLTU, a comparação $4\,294\,967\,295 < 1$ é falsa.
- Na operação SGE, a comparação $-1 \geq 1$ é falsa.
- Na operação SGEU, a comparação $4\,294\,967\,295 \geq 1$ é verdadeira.

Esse exemplo demonstra que a interpretação dos operandos influencia diretamente o resultado das comparações.

5 Testbench e metodologia de testes

Foi desenvolvido um *testbench* denominado `ula_tb`, responsável por aplicar diferentes combinações de operandos e códigos de operação à ULA.

Os estímulos são aplicados sequencialmente, com intervalos de 10 ns entre os testes. Após cada estímulo, o resultado da saída Z é comparado com o valor esperado utilizando instruções `assert`.

Os testes incluem:

- ADD com resultado positivo, negativo e zero;
- SUB com resultado positivo, negativo e zero;
- AND, OR e XOR;
- deslocamentos SLL, SRL e SRA;
- comparações SLT, SLTU, SGE e SGEU;
- comparações SEQ e SNE;
- verificação do sinal `cond`.

5.1 Resultados esperados

Tabela 2 – Casos de teste para operações aritméticas e lógicas

Operação	**Entrada A**	**Entrada B**	**Resultado esperado**
ADD	5	3	8
ADD	5	-5	0
ADD	-5	-3	-8
SUB	10	3	7
SUB	3	5	-2
SUB	5	5	0
AND	10	3	2
OR	10	3	11
XOR	10	3	9

Fonte: Elaboração própria.

Os resultados negativos são representados em complemento de dois na saída de 32 bits.

5.2 Formas de onda

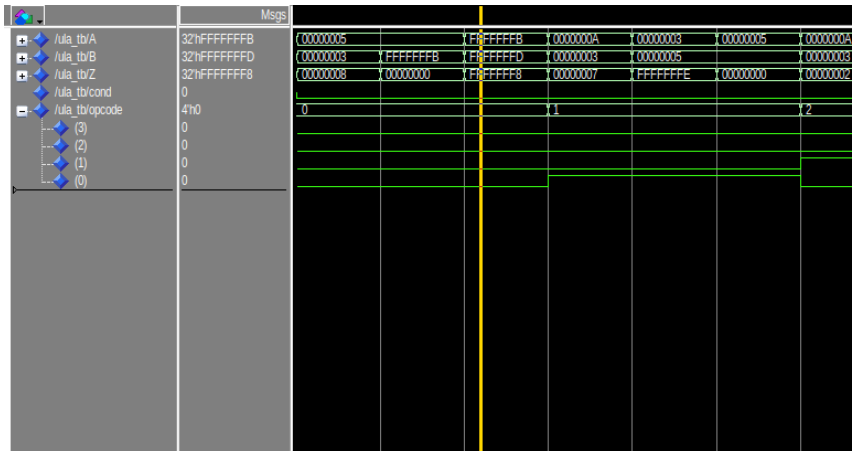


Figura 1 – Formas de onda das operações aritméticas.

Fonte: Simulação da ULA.

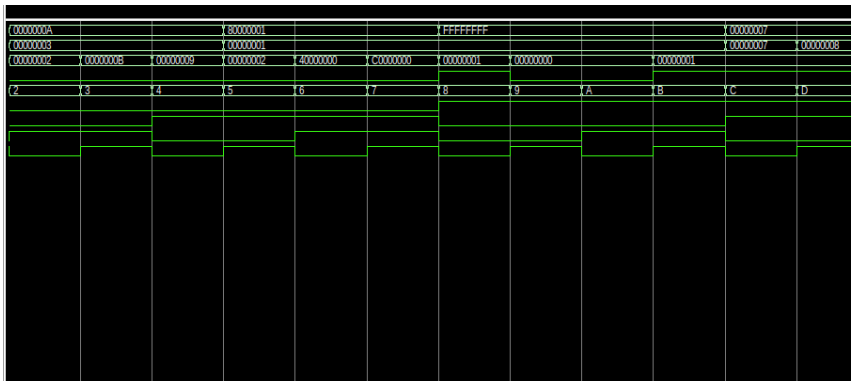


Figura 2 – Formas de onda das operações lógicas e de deslocamento.

Fonte: Simulação da ULA.

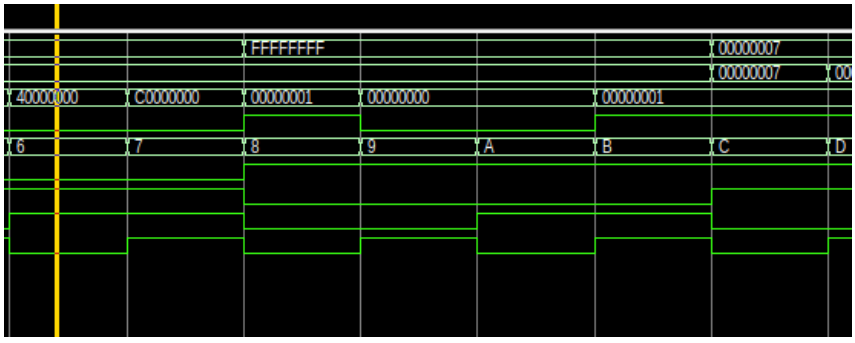


Figura 3 – Formas de onda das operações de comparação.

Fonte: Simulação da ULA.

5.3 Análise dos resultados

A simulação permite comparar os resultados obtidos com os valores esperados para cada operação da ULA. As formas de onda possibilitam analisar os sinais de entrada e saída, enquanto as instruções `assert` identificam possíveis divergências. Dessa forma, é possível verificar o funcionamento das operações aritméticas, lógicas, de deslocamento e de comparação, incluindo o comportamento da saída `cond`.

6 Detecção de overflow nas operações ADD e SUB

Overflow ocorre quando o resultado de uma operação aritmética não pode ser representado na faixa de valores permitida pela interpretação numérica utilizada.

Para números com sinal de 32 bits em complemento de dois, a faixa representável é:

$$-2^{31} \leq x \leq 2^{31} - 1.$$

6.1 Overflow na soma

Na operação ADD, o *overflow* com sinal ocorre quando os dois operandos possuem o mesmo sinal, mas o resultado possui sinal diferente. Considerando A, B e Z como operandos e resultado de 32 bits, o *overflow* pode ser detectado pela expressão:

$$\text{overflow}_{\text{ADD}} = \neg(A_{31} \oplus B_{31}) \wedge (A_{31} \oplus Z_{31}).$$

6.2 Overflow na subtração

Na operação SUB, o *overflow* com sinal ocorre quando os operandos possuem sinais diferentes e o resultado possui sinal diferente do primeiro operando A. A expressão de detecção é:

$$\text{overflow}_{\text{SUB}} = (A_{31} \oplus B_{31}) \wedge (A_{31} \oplus Z_{31}).$$

Nas duas expressões, A_{31} e B_{31} são os bits de sinal dos operandos, enquanto Z_{31} é o bit de sinal do resultado.

Essas expressões verificam *overflow* na interpretação com sinal. Para operações sem sinal, a detecção utiliza o transporte de saída (*carry-out*) na soma e o empréstimo (*borrow*) na subtração.

A implementação apresentada neste trabalho não possui uma saída específica para *overflow*. Portanto, as expressões descrevem como esse recurso poderia ser acrescentado ao circuito.

7 Conclusão

Neste trabalho, foi proposta e implementada uma ULA de 32 bits em VHDL com as operações especificadas no projeto. A implementação utiliza o código de operação para selecionar a função executada e inclui um sinal de condição para as comparações.

O *testbench* foi desenvolvido para verificar as operações aritméticas, lógicas, de deslocamento e de comparação, incluindo resultados positivos, negativos e iguais a zero nas operações aritméticas.

A análise das formas de onda permite observar as mudanças das entradas e saídas durante a simulação e verificar a correspondência entre os resultados produzidos e os valores esperados.

Conclusão baseada na simulação: A simulação permite verificar o funcionamento da ULA e comparar os resultados obtidos com os valores esperados para cada operação. As formas de onda possibilitam observar as mudanças nos sinais de entrada e saída, enquanto as instruções `assert` identificam possíveis divergências. Dessa forma, é possível avaliar se a implementação atende às especificações do projeto.

Referências

[1] Universidade de Brasilia. *Arquitetura e Organização de Computadores: Projeto e Simulação de uma ULA RISC-V em VHDL*. Enunciado da atividade. [S.l.], [s.d.].

[2] IEEE. *IEEE Standard for VHDL Language Reference Manual*. [Completar edição, ano e demais dados bibliográficos consultados.]

[3] IEEE. *Documentação da biblioteca numeric_std utilizada na implementação VHDL*. [Completar com a versão ou fonte efetivamente consultada.]

A Código-fonte da ULA

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ularv is
    generic (
        WSIZE : natural := 32
    );
    port (
        opcode : in  STD_LOGIC_VECTOR(3 downto 0);
        A      : in  STD_LOGIC_VECTOR(WSIZE-1 downto 0);
        B      : in  STD_LOGIC_VECTOR(WSIZE-1 downto 0);
        Z      : out STD_LOGIC_VECTOR(WSIZE-1 downto 0);
        cond   : out STD_LOGIC
    );
end entity ularv;

architecture comportamento of ularv is
begin

    process(opcode, A, B)
        variable resultado : STD_LOGIC_VECTOR(WSIZE-1 downto 0);
        variable condicao  : STD_LOGIC;
        variable shamt     : natural;
    begin
        resultado := (others => '0');
        condicao  := '0';

        -- O RISC-V usa os 5 bits menos significativos de B
        -- para definir o deslocamento de uma palavra de 32 bits.
        shamt := to_integer(unsigned(B(4 downto 0)));

        case opcode is

            -- ADD: A + B
            when "0000" =>
                resultado := STD_LOGIC_VECTOR(
                    unsigned(A) + unsigned(B)
```

```
39                  );
40
41              -- SUB: A - B
42              when "0001" =>
43                  resultado := STD_LOGIC_VECTOR(
44                      unsigned(A) - unsigned(B)
45                  );
46
47              -- AND
48              when "0010" =>
49                  resultado := A and B;
50
51              -- OR
52              when "0011" =>
53                  resultado := A or B;
54
55              -- XOR
56              when "0100" =>
57                  resultado := A xor B;
58
59              -- SLL: deslocamento logico a esquerda
60              when "0101" =>
61                  resultado := STD_LOGIC_VECTOR(
62                      shift_left(unsigned(A), shamt)
63                  );
64
65              -- SRL: deslocamento logico a direita
66              when "0110" =>
67                  resultado := STD_LOGIC_VECTOR(
68                      shift_right(unsigned(A), shamt)
69                  );
70
71              -- SRA: deslocamento aritmetico a direita
72              when "0111" =>
73                  resultado := STD_LOGIC_VECTOR(
74                      shift_right(signed(A), shamt)
75                  );
76
77              -- SLT: menor que, com sinal
78              when "1000" =>
79                  if signed(A) < signed(B) then
```

```
80                      resultado(0) := '1';
81                      condicao := '1';
82                  end if;
83
84              -- SLTU: menor que, sem sinal
85              when "1001" =>
86                  if unsigned(A) < unsigned(B) then
87                      resultado(0) := '1';
88                      condicao := '1';
89                  end if;
90
91              -- SGE: maior ou igual, com sinal
92              when "1010" =>
93                  if signed(A) >= signed(B) then
94                      resultado(0) := '1';
95                      condicao := '1';
96                  end if;
97
98              -- SGEU: maior ou igual, sem sinal
99              when "1011" =>
100                 if unsigned(A) >= unsigned(B) then
101                     resultado(0) := '1';
102                     condicao := '1';
103                 end if;
104
105             -- SEQ: igual
106             when "1100" =>
107                 if A = B then
108                     resultado(0) := '1';
109                     condicao := '1';
110                 end if;
111
112             -- SNE: diferente
113             when "1101" =>
114                 if A /= B then
115                     resultado(0) := '1';
116                     condicao := '1';
117                 end if;
118
119             -- Opcode nao utilizado
120             when others =>
```

```
121                 resultado := (others => '0');
122                 condicao := '0';
123 
124         end case;
125 
126         Z    <= resultado;
127         cond <= condicao;
128 
129     end process;
130 
131 end architecture comportamento;
```

B Código-fonte do testbench

Inserir o código completo do arquivo `ula_tb.vhd`:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity ula_tb is
6  end entity ula_tb;
7
8  architecture teste of ula_tb is
9
10     signal A      : STD_LOGIC_VECTOR(31 downto 0) := (others => '
       0');
11     signal B      : STD_LOGIC_VECTOR(31 downto 0) := (others => '
       0');
12     signal opcode : STD_LOGIC_VECTOR(3 downto 0) := "0000";
13     signal Z      : STD_LOGIC_VECTOR(31 downto 0);
14     signal cond   : STD_LOGIC;
15
16 begin
17
18     DUT: entity work.ularv
19         generic map (
20             WSIZE => 32
21         )
22         port map (
23             opcode => opcode,
24             A      => A,
25             B      => B,
26             Z      => Z,
27             cond   => cond
28         );
29
30     process
31     begin
32
33         -- TESTE 1: ADD positivo: 5 + 3 = 8
34         A <= x"00000005";
```

```
35          B <= x"00000003";
36          opcode <= "0000";
37          wait for 10 ns;
38          assert Z = x"00000008" and cond = '0'
39              report "Erro no ADD positivo" severity error;
40
41          -- TESTE 2: ADD zero: 5 + (-5) = 0
42          A <= x"00000005";
43          B <= x"FFFFFFFB";
44          wait for 10 ns;
45          assert Z = x"00000000"
46              report "Erro no ADD com resultado zero" severity
                error;
47
48          -- TESTE 3: ADD negativo: (-5) + (-3) = -8
49          A <= x"FFFFFFFB";
50          B <= x"FFFFFFFD";
51          wait for 10 ns;
52          assert Z = x"FFFFFFF8"
53              report "Erro no ADD negativo" severity error;
54
55          -- TESTE 4: SUB positivo: 10 - 3 = 7
56          A <= x"0000000A";
57          B <= x"00000003";
58          opcode <= "0001";
59          wait for 10 ns;
60          assert Z = x"00000007"
61              report "Erro no SUB positivo" severity error;
62
63          -- TESTE 5: SUB negativo: 3 - 5 = -2
64          A <= x"00000003";
65          B <= x"00000005";
66          wait for 10 ns;
67          assert Z = x"FFFFFFFE"
68              report "Erro no SUB negativo" severity error;
69
70          -- TESTE 6: SUB zero: 5 - 5 = 0
71          A <= x"00000005";
72          B <= x"00000005";
73          wait for 10 ns;
74          assert Z = x"00000000"
```

```
75              report "Erro no SUB com resultado zero" severity
                    error;
76
77          -- TESTE 7: AND: 10 AND 3 = 2
78          A <= x"0000000A";
79          B <= x"00000003";
80          opcode <= "0010";
81          wait for 10 ns;
82          assert Z = x"00000002"
83              report "Erro no AND" severity error;
84
85          -- TESTE 8: OR: 10 OR 3 = 11
86          opcode <= "0011";
87          wait for 10 ns;
88          assert Z = x"0000000B"
89              report "Erro no OR" severity error;
90
91          -- TESTE 9: XOR: 10 XOR 3 = 9
92          opcode <= "0100";
93          wait for 10 ns;
94          assert Z = x"00000009"
95              report "Erro no XOR" severity error;
96
97          -- TESTE 10: SLL: desloca 80000001 uma posicao a esquerda
98          A <= x"80000001";
99          B <= x"00000001";
100         opcode <= "0101";
101         wait for 10 ns;
102         assert Z = x"00000002"
103             report "Erro no SLL" severity error;
104
105         -- TESTE 11: SRL: deslocamento logico a direita
106         opcode <= "0110";
107         wait for 10 ns;
108         assert Z = x"40000000"
109             report "Erro no SRL" severity error;
110
111         -- TESTE 12: SRA: deslocamento aritmetico a direita
112         opcode <= "0111";
113         wait for 10 ns;
114         assert Z = x"C0000000"
```

```
115             report "Erro no SRA" severity error;
116
117         -- A = -1 com sinal; B = 1
118         A <= x"FFFFFFFF";
119         B <= x"00000001";
120
121         -- TESTE 13: SLT: -1 < 1 e verdadeiro
122         opcode <= "1000";
123         wait for 10 ns;
124         assert Z = x"00000001" and cond = '1'
125             report "Erro no SLT" severity error;
126
127         -- TESTE 14: SLTU: 4294967295 < 1 e falso
128         opcode <= "1001";
129         wait for 10 ns;
130         assert Z = x"00000000" and cond = '0'
131             report "Erro no SLTU" severity error;
132
133         -- TESTE 15: SGE: -1 >= 1 e falso
134         opcode <= "1010";
135         wait for 10 ns;
136         assert Z = x"00000000" and cond = '0'
137             report "Erro no SGE" severity error;
138
139         -- TESTE 16: SGEU: 4294967295 >= 1 e verdadeiro
140         opcode <= "1011";
141         wait for 10 ns;
142         assert Z = x"00000001" and cond = '1'
143             report "Erro no SGEU" severity error;
144
145         -- TESTE 17: SEQ: operandos iguais
146         A <= x"00000007";
147         B <= x"00000007";
148         opcode <= "1100";
149         wait for 10 ns;
150         assert Z = x"00000001" and cond = '1'
151             report "Erro no SEQ" severity error;
152
153         -- TESTE 18: SNE: operandos diferentes
154         A <= x"00000007";
155         B <= x"00000008";
```

```
156         opcode <= "1101";
157         wait for 10 ns;
158         assert Z = x"00000001" and cond = '1'
159             report "Erro no SNE" severity error;
160 
161         report "Todos os testes foram concluidos!"
162             severity note;
163 
164         wait;
165     end process;
166 
167 end architecture teste;
```